



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



INSTITUTO TECNOLÓGICO DE COMITÁN

Av. Instituto tecnológico km 3.5

colonia Yocnajib, el rosario

c.p. 30000

Nombre:

- Luis Alexander Bautista Sánchez
- Brayant Jared Dominguez Montero

Números de control:

- A23700614
- CA18887059

Carrera:

Ingeniería en sistemas computacionales.

Semestre:

6

Grupo:

Único.

Materia:

LENGUAJES Y AUTOMATAS II

Nombre del profesor:

ING. HUGO ANTONIO OCAMPO ALFARO

Trabajo:

compilador astrax

Lugar y fecha: Comitán de Domínguez Chiapas a 23 Mayo 2026.

INDICE.

INTRODUCCIÓN:	3
DEFINICIÓN DEL LENGUAJE	5
TIPOS DE DATOS	5
OPERADORES ARITMÉTICOS	5
OPERADORES RELACIONALES	5
ESTRUCTURAS DE CONTROL	5
LOGO ASTRAX	6
PALABRAS RESERVADAS	7
SÍMBOLOS SOPORTADOS	7
REGLAS DE SINTAXIS	8
SALIDA DE DATOS	9
CAPTURA DE DATOS	9
ESTRUCTURAS DE CONTROL DE FLUJO	10
CONDICIONALES	10
CICLOS Y REPETICIONES	11
Ciclo FOR (orbitx)	11
Ciclo While (orbit)	12
PROCESO DEL CICLO	12
Diferencias entre ciclo FOR y ciclo WHILE	13
FOR (orbitx)	13
WHILE (orbit)	13
ARREGLOS DINÁMICOS	14
MODULARIDAD	14
ENTORNO DE DESARROLLO	15
LINK DE DESCARGA	
https://www.mediafire.com/file/0l6mkbw403u642a/Astrax_setup.zip/file	15
Conclusión	16

INTRODUCCIÓN:

Astrax se caracteriza por ser un lenguaje interpretado, dinámico y de alto nivel, diseñado para facilitar el aprendizaje sin perder solidez técnica. Además, incorpora elementos como análisis léxico, generación de árboles de sintaxis y un intérprete en tiempo real, permitiendo una ejecución eficiente del código. Su diseño busca no solo resolver problemas computacionales, sino también transmitir una identidad visual y conceptual basada en el universo, las estrellas y el movimiento cósmico.

Este documento reúne los aspectos más importantes de Astrax, incluyendo sus palabras reservadas, estructuras de control, manejo de variables, funciones, arreglos y entorno de desarrollo, resaltando tanto su funcionamiento lógico como su propuesta creativa.

El desarrollo de este lenguaje surge como una propuesta orientada a demostrar los conocimientos adquiridos en las áreas relacionadas con compiladores, intérpretes, análisis sintáctico y diseño de lenguajes de programación. A través de Astrax se busca representar de manera práctica cómo un lenguaje puede ser construido desde sus fundamentos, definiendo reglas de sintaxis, estructuras de control y mecanismos de ejecución que permitan al usuario interactuar con él de forma sencilla e intuitiva.

Uno de los principales objetivos del proyecto es ofrecer una experiencia de programación accesible para estudiantes y usuarios que están iniciando en el mundo del desarrollo de software. Por esta razón, la sintaxis fue diseñada para ser clara y fácil de comprender, utilizando palabras reservadas inspiradas en conceptos astronómicos que aportan originalidad sin afectar la funcionalidad del lenguaje. Esta combinación entre creatividad y aspectos técnicos permite que el aprendizaje resulte más atractivo y dinámico.

Asimismo, Astrax incorpora elementos fundamentales presentes en los lenguajes modernos, como la declaración dinámica de variables, estructuras condicionales, ciclos iterativos, manejo de arreglos y modularidad mediante funciones. Estas características permiten desarrollar programas capaces de resolver diferentes problemas computacionales, manteniendo una organización adecuada del código y

promoviendo buenas prácticas de programación.

Otro aspecto importante es la implementación de un entorno de desarrollo propio, el cual complementa la experiencia del usuario mediante herramientas de productividad y depuración. Gracias a ello, el programador puede escribir, ejecutar y corregir sus programas de forma más eficiente, comprendiendo mejor el comportamiento del lenguaje y los procesos internos que ocurren durante la ejecución.

Finalmente, Astrax representa la integración de conocimientos teóricos y prácticos aplicados en un proyecto funcional. Más allá de ser únicamente un simulador o una propuesta conceptual, constituye un ejemplo del proceso completo de diseño de un lenguaje de programación, desde la definición de sus reglas hasta la interpretación de instrucciones. De esta manera, el proyecto demuestra cómo la innovación, la creatividad y la lógica computacional pueden combinarse para construir una herramienta educativa capaz de facilitar el aprendizaje y fortalecer las habilidades de desarrollo de software.

DEFINICIÓN DEL LENGUAJE.

- Alfanumérico.
- Lista de palabras reservadas.
- Ejecución por medio de AST

TIPOS DE DATOS.

- Enteros (positivos, negativos).
- Texto (String).
- Booleanos (implícitos)

OPERADORES ARITMÉTICOS.

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/).

OPERADORES RELACIONALES.

- Igual (==)
- Diferente (!=)
- Mayor que (>)
- Menor que (<)
- Mayor o igual que (>=)
- Menor o igual que (<=)

ESTRUCTURAS DE CONTROL.

- Condicional (if/else)
 - nova (condición)
 - Instrucciones.
- void:
 - Instrucciones.
- halt
- salidas:
 - Pulse valor, acepta números y texto. -
- Ciclo while.

orbit (condicion)

Instrucciones.

halt

orbitx (zen variable flux valor inicial; condición;
actualización). halt.

LOGO ASTRAX.



El logo de Astrax fue diseñado para representar la esencia y la identidad del lenguaje de programación, tomando inspiración directa del universo, las estrellas y los fenómenos cósmicos. Su propósito no es únicamente funcionar como un elemento visual, sino también transmitir los valores principales del proyecto: innovación, exploración, creatividad y conocimiento tecnológico. Además, la identidad visual de Astrax está relacionada con la temática espacial presente en sus palabras reservadas y estructuras de programación, tales como *orbit*, *nova*, *quasar* y *nebula*. Esta relación permite que exista una coherencia entre la apariencia gráfica del proyecto y el funcionamiento interno del lenguaje, fortaleciendo su personalidad y diferenciándolo de otros entornos de programación tradicionales.

PALABRAS RESERVADAS

Palabra	Categoría	Descripción / Uso Principal
<code>zen</code>	Memoria	Declaración o instanciación de nuevas variables.
<code>flux</code>	Operador	Operador de asignación de flujo (equivalente a "=").
<code>pulse</code>	I/O	Emisión de salida de datos a la consola.
<code>quasar</code>	I/O	Captura de entrada de datos interactiva del usuario.
<code>nova / void</code>	Control	Estructura condicional (If / Else).
<code>orbit</code>	Iteración	Bucle de condición indefinida (While).
<code>orbitx</code>	Iteración	Bucle de condición determinada (For).
<code>nebula</code>	Modularidad	Declaración de funciones reutilizables.
<code>core</code>	Modularidad	Instrucción de retorno de valores funcionales.
<code>halt</code>	Estructural	Clausura obligatoria de bloques lógicos (nova, orbit, nebula).

SÍMBOLOS SOPORTADOS.

- ; (Fin de instrucción)
- () (Paréntesis)
- : (Inicio de texto)
- “ ” (Delimitador de texto)
- +,-,*,/ (operadores)
- = (Asignación) Flux
- >,<, >=, <=, ==, != (comparadores).

REGLAS DE SINTAXIS.

El lenguaje maneja la inferencia de tipos automáticamente. No es necesario definir si un dato es un número o texto; el intérprete lo resuelve en tiempo de ejecución. Toda sentencia en Astrax debe finalizar con punto y coma (;).

Declaración y Asignación Dinámica.

Para reservar espacio, se utiliza **zen** , el identificador y **flux** .

La siguiente imagen muestra cómo Astrax permite declarar variables de manera dinámica utilizando la palabra reservada **zen** y el operador de asignación **flux**. Una de las características principales del lenguaje es que no requiere que el programador especifique explícitamente el tipo de dato, ya que el intérprete es capaz de identificarlo automáticamente durante la ejecución del programa.

En la primera línea se declara una variable llamada **entero** con el valor numérico **10**. Como el valor no contiene decimales, Astrax interpreta automáticamente que se trata de un número entero.

En la segunda línea se crea una variable llamada **flotante** con el valor **99.99**. Debido a que el número contiene parte decimal, el intérprete reconoce que corresponde a un dato de tipo flotante o decimal.

Finalmente, en la tercera línea se declara la variable **texto**, la cual almacena la cadena de caracteres **"Sistema Operativo"**. Las cadenas de texto deben colocarse entre comillas dobles para que el lenguaje las identifique correctamente como información textual.

```
// Tipos de datos nativos soportados
zen entero flux 10;
zen flotante flux 99.99;
zen texto flux "Sistema Operativo";
```

- Los bloques usan : y halt.
- Las condiciones usan ().
- Las variables se declaran como zen.
- La palabra reservada quasar permite solicitar y capturar datos ingresados por

el usuario.

SALIDA DE DATOS.

La instrucción `pulse` evalúa cualquier expresión y la envía al `stdout` (consola).

Soporta concatenación nativa de texto y números utilizando el operador `+`.

```
zen costo flux 500;  
pulse "El costo total del servicio es: $" + costo;
```

La salida de datos en Astrax se realiza mediante la palabra reservada `pulse`, cuya función es mostrar información en la consola para que el usuario pueda visualizar resultados, mensajes o valores almacenados en variables. Esta es una de las instrucciones más utilizadas dentro del programa ya que permite verificar el comportamiento del código y presentar información de manera clara.

En la primera línea se declara una variable llamada `costo` y se le asigna el valor de 500 mediante el operador `flux`.

Posteriormente, la instrucción `pulse` muestra un mensaje en pantalla. Combina un texto con el valor almacenado en la variable `costo` utilizando el operador `+`, el cual permitirá realizar una concatenación entre cadenas de texto y datos numéricos.

CAPTURA DE DATOS.

La instrucción `quasar` permite generar scripts interactivos. Recibe una expresión (que funciona como el mensaje en pantalla) y el nombre de la variable donde se almacenará el dato ingresado.

```
zen nombre_cliente flux "  
quasar "Ingrese el nombre del cliente: " nombre_cliente;  
pulse "Cliente " + nombre_cliente + " registrado.";
```

La captura de datos en Astrax se realiza mediante la palabra reservada **quasar**, la cual permite solicitar información al usuario durante la ejecución del programa. Esta característica hace posible crear aplicaciones interactivas, donde el usuario puede

ingresar datos que posteriormente serán utilizados por el sistema para realizar operaciones o mostrar resultados personalizados.

La estructura general para capturar datos en Astrax es la siguiente.

- `quasar`, es la instrucción encargada de solicitar datos al usuario.
- “Mensaje para el usuario” corresponde al texto que se mostrara en pantalla como guía.
- `Variable` es el identificador donde se almacenara la información ingresada.
- `;` indica el final de la instrucción.

La captura de datos es una de las funcionalidades más importantes dentro de cualquier lenguaje de programación, ya que permite desarrollar programas dinámicos e interactivos.

`quasar` es la instrucción que simplifica el proceso en Astrax, al ofrecer una sintaxis clara y fácil de comprender, favoreciendo el aprendizaje y la creación de aplicaciones más cercanas a las necesidades del usuario.

ESTRUCTURAS DE CONTROL DE FLUJO.

CONDICIONALES.

La toma de decisiones se construye con `nova` (verdadero) y `void` (falso). Todo bloque de decisión debe terminar con `halt`.

```
zen ancho_banda flux 50;

nova (ancho_banda >= 100) :
    pulse "Conexión de alta velocidad.";
void :
    pulse "Requiere actualización de plan.";
halt
```

CICLOS Y REPETICIONES.

Astrax soporta dos paradigmas de iteración: ciclos condicionales (orbit) y ciclos indexados (orbitx).

```
// Ciclo Determinado (FOR)
orbitx (zen i flux 1; i <= 5; i flux i + 1) :
    pulse "Procesando paquete " + i;
halt

// Ciclo Indeterminado (WHILE)
zen bateria flux 100;
orbit (bateria > 0) :
    bateria flux bateria - 20;
halt
```

En Astrax existen dos tipos principales de ciclos o estructuras repetitivas:

- orbitx (ciclo FOR) determinado.
- orbit (ciclo WHILE) indeterminado.

Ambos sirven para repetir instrucciones varias veces, pero se utilizan en situaciones diferentes.

Ciclo FOR (orbitx)

El ciclo FOR se usa cuando sabemos cuántas veces queremos repetir una acción. El orbitx tiene 3 partes importantes en el ejemplo.

1.- Inicio.

</> astrax

zen i flux 1;

Aquí es donde se crea la variable i con valor inicial de 1.

2.- Condición

</> astrax

i<=5;

El ciclo continuará mientras i sea menor o igual a 5.

3.- Incremento.

</> astras

i flux i + 1

Después de cada repetición, i aumenta 1

El ciclo FOR imprime un mensaje en pantalla en cada repetición. Se usa cuando se sabe el número exacto de repeticiones, o se va a correr listas o arreglos.

Ciclo While (orbit)

El ciclo WHILE se usa cuando no se sabe exactamente cuántas veces se repetirá algo.

Funciona con una variable inicial

</> astrax

zen batería flux 100;

La batería comienza en 100

Condición del ciclo.

</> astrax

orbit (batería > 0):

El ciclo seguirá ejecutándose mientras la batería sea mayor que cero. Acción repetitiva.

</> astrax

bateria flux batería -20;

En cada repetición se restan 20 a la batería.

PROCESO DEL CICLO.

Repetición	Batería
Inicio	100
1	80
2	60
3	40
4	20
5	0

Cuando llega a 0, la condición deja de cumplirse y el ciclo termina. Se usa cuando no se sabe cuántas veces se repetirá algo, y también se depende de una condición, o a la espera de datos o eventos.

Diferencias entre ciclo FOR y ciclo WHILE.

FOR (orbitx)

Repeticiones definidas

Usa contador

Ideal para contar

Más estructurado.

WHILE (orbit)

Repeticiones indefinidas

Usa condicion

Ideal para esperar condiciones.

Más flexible.

En resumen FOR es para cuando sabemos cuantas veces repetir y WHILE cuando dependemos de una condición.

ARREGLOS DINÁMICOS.

Los arreglos dinámicos en Astrax permiten almacenar múltiples valores dentro de una sola variable, facilitando la organización y manipulación de información relacionada. Cada elemento del arreglo ocupa una posición específica llamada **índice**, la cual comienza en **0**, siguiendo el estándar utilizado en la mayoría de los lenguajes de programación modernos. .

```
zen ips flux ["192.168.1.1", "10.0.0.1"];
pulse "IP Principal: " + ips[0];

// Modificación de índice
ips[1] flux "10.0.0.254";

// Fusión de arreglos mediante operador suma
zen nuevas_ips flux ["172.16.0.1"];
zen red_global flux ips + nuevas_ips;
```

Este ejemplo demuestra características fundamentales de los arreglos en Astrax.

- Almacenamiento de múltiples datos en una sola estructura.
- Acceso a elementos mediante índices.
- Modificación dinámica de valores ya existentes.
- combinación o fusión de arreglos para crear nuevas colecciones de datos.

Los arreglos dinámicos representan una herramienta fundamental dentro de Astrax, ya que permiten gestionar conjuntos de información de forma eficiente y organizada.

MODULARIDAD.

La modularidad es una característica fundamental de Astrax, ya que permite dividir un programa en bloques de código independientes llamados **nebula** (funciones). Gracias a este mecanismo, es posible reutilizar instrucciones, mejorar la organización del código y facilitar el mantenimiento de los programas.

Una función puede recibir datos de entrada, procesarlos internamente y devolver un resultado al programa principal. Además, las variables creadas dentro de una **nebula** existen únicamente durante la ejecución de dicha función, evitando interferencias con otras partes del sistema y garantizando una mejor administración de la memoria.

```
nebula aplicar_descuento(monto, descuento) :  
    zen ahorro flux monto * (descuento / 100);  
    core monto - ahorro;  
halt  
  
zen total flux aplicar_descuento(1200, 15);  
pulse "Total a facturar: $" + total;
```

La modularidad permite construir programas más organizados, reutilizables y fáciles de comprender. En lugar de repetir el mismo código varias veces, las funciones permiten encapsular procesos específicos y ejecutarlos cuando sea necesario. Esta característica resulta especialmente útil en proyectos grandes, donde la división del código en módulos mejora la legibilidad, reduce errores y facilita futuras modificaciones o ampliaciones del sistema.

ENTORNO DE DESARROLLO.

El lenguaje viene empaquetado con su propio IDE de temática Cyberpunk, optimizado para productividad estructural: CyberGraph Monitor: Gráfico vectorial superior que muestrea la actividad del CPU en tiempo real, ideal para detectar cuellos de botella en ciclos infinitos. Sistema Snippet-Tab: Inyección de plantillas automáticas. Al escribir orbitx , orbit , nova o nebula y presionar la tecla Tabulador, el editor autocompleta toda la estructura jerárquica y posiciona el cursor. Consola de Depuración: Emite mensajes de excepción exactos, indicando la línea y el símbolo que originó el error sintáctico o matemático (ej. índices fuera de rango).

LINK DE DESCARGA

https://www.mediafire.com/file/016mkbw403u642a/Astrax_setup.zip/file

Conclusión

En conclusión, Astrax representa una propuesta innovadora dentro del ámbito de los lenguajes de programación, al combinar fundamentos sólidos de arquitectura de software con una temática astronómica altamente creativa. Su estructura organizada, sintaxis intuitiva y herramientas integradas permiten desarrollar programas de manera clara, eficiente y modular.

Además de sus capacidades técnicas, Astrax destaca por transformar conceptos complejos de programación en elementos visuales y simbólicos inspirados en el cosmos, facilitando el aprendizaje y haciendo más atractiva la experiencia del programador. La integración de variables dinámicas, estructuras de control, funciones reutilizables y arreglos demuestra que el lenguaje posee características modernas y funcionales para distintos tipos de proyectos.

En general, este proyecto refleja cómo la creatividad y la tecnología pueden fusionarse para construir herramientas innovadoras, demostrando que programar también puede entenderse como la creación y organización de un universo lógico propio.

Durante el desarrollo de este trabajo fue posible comprender con mayor profundidad los procesos que intervienen en la creación de un lenguaje de programación, desde la definición de reglas léxicas y sintácticas hasta la ejecución de instrucciones mediante un intérprete basado en árboles de sintaxis abstracta (AST). Esto permitió aplicar conocimientos teóricos vistos en clase y transformarlos en una solución práctica capaz de simular el comportamiento de un lenguaje funcional.

Asimismo, el proyecto permitió reconocer la importancia de diseñar una sintaxis clara y coherente para mejorar la interacción entre el usuario y el sistema. Cada palabra reservada, estructura de control y elemento del lenguaje fue pensada para mantener una relación con la temática espacial de Astrax, logrando una identidad propia que lo diferencia de otros lenguajes tradicionales sin perder su utilidad técnica.

Otro aspecto relevante fue la implementación de herramientas que favorecen la productividad y la detección de errores, demostrando que un lenguaje no solo depende de sus instrucciones internas, sino también de los recursos que ofrece para facilitar el trabajo del programador. La incorporación de un entorno de desarrollo personalizado contribuye a mejorar la experiencia de uso y fortalece el aprendizaje de quienes utilizan el sistema.

Finalmente, Astrax demuestra que el diseño de software puede ir más allá de la resolución de problemas computacionales y convertirse también en un ejercicio de innovación y creatividad. El proyecto evidencia la capacidad de integrar conocimientos de programación, compiladores, estructuras de datos y diseño de interfaces en una propuesta completa y funcional. Como resultado, se obtiene una herramienta que no solo cumple con los objetivos académicos planteados, sino que también sirve como base para futuras mejoras y expansiones, permitiendo incorporar nuevas características que continúen enriqueciendo el lenguaje y ampliando sus posibilidades de aplicación.